Tan Ngo

+84 364-143-275 — Email — Personal Page — LinkedIn — GitHub

Education

Hanoi University of Science and Technology (HUST) Aug 2021 – Apr 2025
B.Sc. in Computer Science, Hanoi, Vietnam

- GPA: 3.69/4.0; graduated early in 3.5 years and recognized as one of 4 outstanding graduating students (Director's Certificate of Merit).
- Selected to deliver the graduation speech on behalf of all graduating students.

Lam Son High School for the Gifted — *Specialized Mathematics Program, Thanh Hoa, Vietnam* Aug 2018 – May 2021

- **Vietnam National Mathematics Competition, Second Prize — TST List** (2021) — Ranked 11th nationally and selected for the Team Selection Test, the second round for selecting Vietnam's International Mathematical Olympiad team.
- **Vietnam National Mathematics Competition, Consolation Prize** (2020) — National award in Vietnam's highest-level mathematics competition for gifted high school students.
- **North Central Region Mathematics Olympiad, First Prize** (2019) — Among specialized high schools in the North Central Region of Vietnam.

Publications

Track the Noise, Move the World: 3D-Grounded Motion-Consistent Noise for Controllable Video Generation

- Long Van Vu[*], **Tan Van Ngo**[*], Animesh Karnewar, Amir Habibian, Binh-Son Hua, Hung H. Bui, Minh Hoai, and Phong Ha Nguyen. *Under review at NeurIPS 2026.* · [*]**Co-first author** (equal contribution). · arXiv:2607.02798

Research and Work Experience

AI Research Resident, Qualcomm AI Research Vietnam Jun 2025 – Present
Supervisors: Phong Ha Nguyen, Ph.D. and Binh-Son Hua, Ph.D. (see References).

– **Unifying Camera and Motion Control for Video Generation** (Jan 2026 – Present)

- Developing a method to jointly control **camera movement and object motion** in a video diffusion model, so a user can specify both the trajectory of the scene and the movement of subjects within it — a capability prior work typically handles only one of at a time.
- Introducing **depth estimation** as an explicit 3D signal so the model reasons about scene geometry, rather than relying on 2D cues alone, to improve motion consistency across frames.
- Scaling training, inference, and evaluation across **multi-node GPU clusters (up to 300 GPUs)** on RUNAI, working on data preparation, synchronization, throughput, and training stability.
- **Fixed a model-loading default that made all 96 GPUs (12 nodes × 8) queue behind one another at startup**, re-downloading weights already on disk: per-node startup **9 min** → **∼44 s (∼12×)**, GPU start spread within a node **267 s** → **∼11 s**, a recurring multi-node crash removed, inference throughput unchanged.

– **Self-Directed Systems Research** (2026 – Present)

- **Favourite courses & self-study — ML systems from first principles: Stanford CS336 · Karpathy's *Zero to Hero* · tinytorch · tinygrad · GPU-MODE.**
- **Customized Ray to make my own research loop 5.4× faster.** Rebuilt the evaluation pipeline for my video project (**CLIP, optical-flow EPE, LPIPS, SSIM, PSNR, FID, FVD**) as a **pool of workers pulling from a shared queue**, replacing a shell launcher that split work unevenly and silently skipped data, and combined results with a **tree-reduction I wrote myself**. Scores matched the original **to within 1.79e-7 per video** (unchanged) while running **5.4× faster on 8 GPUs**.
- **Measured where Ray's defaults quietly cost performance:** each worker gets a **single CPU thread** by default, slowing CPU-bound work **4–9×**; coordinating work centrally rather than inside the workers costs **∼30×** the actual work on small transfers. **Ray's transport itself adds no overhead — the cost is in the defaults**, which is how performance is lost with no visible error.

- **Found that adding GPUs can make a pipeline slower:** my evaluation ran fastest on **2 of 8 GPUs** and the depth/pose annotation on **4 of 8**, because the container's CPU budget is capped (**198 cores**) and is split among workers — past a point each extra GPU only waits for CPU. **I now measure the best worker count instead of using every GPU available.**

– **Few-Step Text-to-Multiview Diffusion Model** (Jun 2025 – Dec 2025)
- Distilled a multi-step **MVDream-style** text-to-multiview diffusion model into a **few-step** student, sharply reducing the number of sampling steps while preserving multiview consistency, to make generation fast enough for interactive and on-device use.
- Raised generation from **256×256 to 512×512** per view — **4× the pixels**, above the resolution the teacher itself produced — and improved output quality through training-recipe changes, data refinement, and model optimization.
- **Quantized** the few-step model and supported the applied team in deploying it on **Samsung S24** phones; the result was shown as a live demo at Qualcomm's NeurIPS 2025 booth.

Generative AI Engineer, ZenAI Aug 2024 – Jun 2025

– **Diffusion Image-Generation Acceleration (FLUX, SDXL)** (Aug 2024 – Jun 2025)
- Sped up image generation for **FLUX** and **SDXL** diffusion models in production:
  - **FLUX.1-schnell (8-step):** combined **context parallelism (2× H100)**, `torch.compile` **(TorchInductor)** op fusion, and **first-block caching** to cut end-to-end inference from **2.02 s (1× H100 baseline) → 0.86 s (≈2.35× faster)** while preserving **≥95% image quality**.
  - **SDXL (DreamShaper-XL v2 Turbo, 8-step, 1024×1024):** combined **CFG parallelism (2× H100)** and `torch.compile` to cut inference from **1.26 s (1× H100 baseline) → 0.72 s (≈1.75× faster)** while preserving **≥99% image quality**.
- **Root-caused a bug inside PyTorch's compiler that was blocking these speedups entirely** (torch 2.3.1): the compiler aborted with an error pointing nowhere near the cause, so I instrumented the failing stage to make an opaque symbolic pipeline readable, isolated the **single operation whose index expression could not be built**, and added a **fallback** so such an operation degrades instead of killing the whole compilation — restoring the compiled path.
- **Found the limit of the technique, not just its benefit:** `torch.compile`'s speedup holds only while the model sees a **small number of distinct input shapes (∼16)** — it degrades as shape variety grows, and at **∼100 shapes falls back to uncompiled speed**, silently, because each new shape forces a recompilation until the compiler stops specializing. A fixed-resolution benchmark therefore reports a win production never sees; the fix is to **serve a small bucketed set of shapes** rather than compile per shape.

– **LLM Serving & Inference Optimization (vLLM, SGLang)** (Aug 2024 – Jun 2025)
- Served and optimized **large language models** with the **vLLM** and **SGLang** frameworks for production workloads.
- Applied **quantization**, **KV-cache optimization**, and **parallel GPU execution** to cut latency and raise throughput under production serving loads.
- Reduced **CPU–GPU communication overhead** to improve end-to-end serving efficiency.

Research Assistant, Computer Vision Lab, BKAI, HUST Oct 2023 – Jan 2025

– **Continual Learning for Object Detection with Generative Data Replay** (Sep 2024 – Jan 2025)
- Applied **continual learning** to object detection to mitigate catastrophic forgetting as new tasks/classes arrive sequentially, instead of retraining on all data.
- Used **diffusion-generated replay data** to stand in for past-task samples, improving knowledge retention and training stability without storing the original datasets.

– **Vision Transformers for Medical Endoscopy Image Segmentation** (Oct 2023 – Aug 2024)
- Researched **Vision Transformer**–based architectures for segmentation of medical **endoscopy** images, using **Masked Autoencoder (MAE)** self-supervised pretraining on unlabeled data to improve downstream quality under limited labels.

Honors and Awards

HUST Wings Scholarship — Awarded for six consecutive semesters. 2022–2024

HUST Academic Encouragement Scholarship — Computer Science department, <3% selected. 2021–2022

Technical Skills

Languages: Python, C/C++
Distributed & parallel: multi-GPU / multi-node training and inference (up to 300 GPUs), NCCL, CPU/GPU workload balancing, Ray, SLURM
Efficiency & compilers: `torch.compile`, quantization, KV-cache, distillation, vLLM, SGLang
Frameworks & libraries: PyTorch, Diffusers, NumPy, OpenCV

References

Binh-Son Hua, Ph.D.
Assistant Professor, Trinity College Dublin; Consultant, Qualcomm AI Research Vietnam.
E-mail: binhson.hua@tcd.ie / binhson.hua@gmail.com
Website: sonhua.github.io

Phong Ha Nguyen, Ph.D.
Senior Research Scientist, Qualcomm, Hanoi, Vietnam.
Ph.D. in Computer Science, University of Oulu, Finland.
E-mail: phongnhhn92@gmail.com / phongnh@qti.qualcomm.com
Website: phongnhhn.info